Name: Menna Hesham Ali – Group: SE2

Local Serverless Computing and Event-Driven Image Processing Pipeline

5. Create the Project Structure:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> tree /F
Folder PATH listing for volume data
Volume serial number is 5423-43ED
D:.
    docker-compose.yml

├──data
    ├──input
    └──output
├──event_source
        Dockerfile
        requirements.txt
        watcher.py

├──functions
    ├──image_resizer
            app.py
            Dockerfile
            requirements.txt

    └──notifier
            app.py
            Dockerfile
            requirements.txt

└──router
        Dockerfile
        event_router.py
        requirements.txt
```

From 6 to 10 - Project Setup Completion:

All required project files and directories were successfully created, including the Event Source, Event Router, and both function services (Image Resizer and Notifier).

11. Step 6 - Run the Full Local Pipeline:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose up
Attaching to lab5-event-router, lab5-event-source, lab5-image-resizer, lab5-notifier, lab5-redis
lab5-redis  | 1:C 02 May 2026 22:27:57.075 * oO0OoO0OoO0Oo Redis is starting oO0OoO0OoO0Oo
lab5-redis  | 1:C 02 May 2026 22:27:57.080 * Redis version=7.4.8, bits=64, commit=00000000, modified=0, pid=1, just star
ted
lab5-redis  | 1:C 02 May 2026 22:27:57.080 # Warning: no config file specified, using the default config. In order to sp
ecify a config file use redis-server /path/to/redis.conf
lab5-redis  | 1:M 02 May 2026 22:27:57.080 * monotonic clock: POSIX clock_gettime
lab5-redis  | 1:M 02 May 2026 22:27:57.089 * Running mode=standalone, port=6379.
lab5-redis  | 1:M 02 May 2026 22:27:57.090 * Server initialized
lab5-redis  | 1:M 02 May 2026 22:27:57.106 * Ready to accept connections tcp
lab5-notifier  |  * Serving Flask app 'app'
lab5-notifier  |  * Debug mode: off
lab5-notifier  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.


lab5-notifier       |  * Running on all addresses (0.0.0.0)


lab5-image-resizer  |  * Debug mode: off
lab5-notifier       |  * Running on http://127.0.0.1:5000
lab5-image-resizer  | WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
lab5-notifier       |  * Running on http://127.0.0.1:5000

lab5-image-resizer  |  * Running on all addresses (0.0.0.0)


lab5-image-resizer  |  * Running on http://127.0.0.1:5000
lab5-notifier       | Press CTRL+C to quit
```

In this step, the entire system was executed using Docker Compose:

- The docker compose up --build command was used to build and start all services, including Redis, Event Source, Event Router, Image Resizer, and Notifier.
- The system successfully initialized, and logs confirmed that each service was running and ready to process events.

12. Step 7 - Create a Test Image Locally:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> $script = @"
>> from PIL import Image, ImageDraw
>>
>> img = Image.new("RGB", (1200, 800), (240, 240, 240))
>> draw = ImageDraw.Draw(img)
>>
>> draw.text((50, 50), "Lecture 5 Local Serverless Lab", fill=(0, 0, 0))
>> draw.text((50, 100), "Event-driven image processing pipeline", fill=(0, 0, 0))
>>
>> img.save("/data/input/test_image.png")
>>
>> print("Created /data/input/test_image.png")
>> "@
```

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> $script | docker compose exec -T image-resizer python
Created /data/input/test_image.png
```

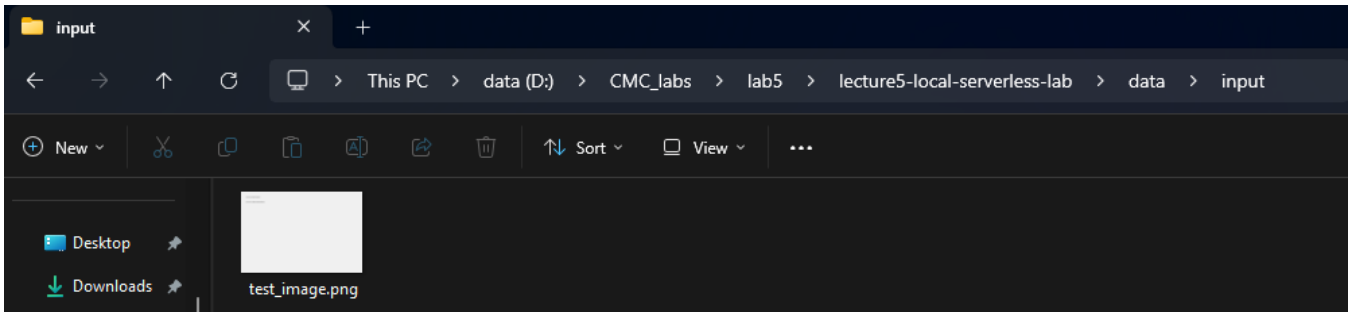
A test image was generated inside the container and stored in **/data/input**, triggering the event-driven pipeline.

```
lab5-event-source   | Event Source started. Watching folder: /data/input
lab5-event-source   | Published event: {
lab5-event-source   |   "event_id": "db615c4a-8559-4b8c-b90a-cc877929e2ed",
lab5-event-source   |   "event_type": "image.uploaded",
lab5-event-source   |   "file_name": "test_image.png",
lab5-event-source   |   "file_path": "/data/input/test_image.png",
lab5-event-source   |   "width": 300,
lab5-event-source   |   "created_at": "2026-05-02T22:54:46.558814"
lab5-event-source   | }
lab5-event-router   | Event Router started. Waiting for events...
lab5-event-router   |
lab5-event-router   | Received event: image.uploaded
lab5-event-router   | {
lab5-event-router   |   "event_id": "db615c4a-8559-4b8c-b90a-cc877929e2ed",
lab5-event-router   |   "event_type": "image.uploaded",
lab5-event-router   |   "file_name": "test_image.png",
lab5-event-router   |   "file_path": "/data/input/test_image.png",
lab5-event-router   |   "width": 300,
lab5-event-router   |   "created_at": "2026-05-02T22:54:46.558814"
lab5-event-router   | }


lab5-image-resizer  | 172.20.0.6 - - [02/May/2026 22:54:48] "POST /resize HTTP/1.1" 200 -
lab5-notifier       | NOTIFICATION: {'status': 'notified', 'function': 'notifier', 'event_type': 'image.uploaded', 'file
_name': 'test_image.png', 'timestamp': '2026-05-02T22:54:48.483055'}
lab5-event-router   | Sent to http://image-resizer:5000/resize
lab5-event-router   | Response code: 200
```

This proves:

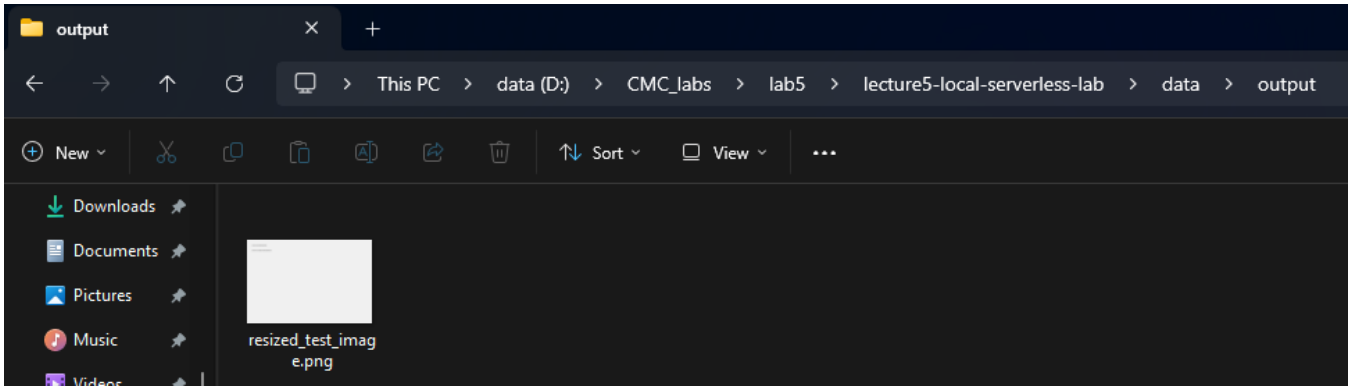
- The system detected the image
- Event was sent to Redis



A test image was generated inside the container using a Python script and saved to the /data/input directory.

13. Step 8 - Check the Output:

```
lab5-event-router   | Duration: 1823.14 ms
lab5-event-router   | Response: {"function":"image-resizer","input_file":"test_image.png","new_size":[300,200],"original
_size":[1200,800],"output_file":"/data/output/resized_test_image.png","processing_ms":1809.32,"status":"success"}
lab5-event-router   |
lab5-event-router   | Sent to http://notifier:5000/notify
lab5-event-router   | Response code: 200
lab5-event-router   | Duration: 9.73 ms
lab5-event-router   | Response: {"event_type":"image.uploaded","file_name":"test_image.png","function":"notifier","statu
s":"notified","timestamp":"2026-05-02T22:54:48.483055"}
lab5-event-router   |
lab5-redis          | 1:M 02 May 2026 23:54:45.019 * 1 changes in 3600 seconds. Saving...
lab5-redis          | 1:M 02 May 2026 23:54:45.024 * Background saving started by pid 22
lab5-redis          | 22:C 02 May 2026 23:54:45.050 * DB saved on disk
lab5-redis          | 22:C 02 May 2026 23:54:45.051 * Fork CoW for RDB: current 0 MB, peak 0 MB, average 0 MB
lab5-redis          | 1:M 02 May 2026 23:54:45.125 * Background saving terminated with success
```



The successful generation of resized_test_image.png in the /data/output directory confirms that the entire event-driven pipeline executed correctly.

Check logs:

- docker compose logs event-source

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose logs event-source
lab5-event-source   | Event Source started. Watching folder: /data/input
lab5-event-source   | Published event: {
lab5-event-source   |   "event_id": "db615c4a-8559-4b8c-b90a-cc877929e2ed",
lab5-event-source   |   "event_type": "image.uploaded",
lab5-event-source   |   "file_name": "test_image.png",
lab5-event-source   |   "file_path": "/data/input/test_image.png",
lab5-event-source   |   "width": 300,
lab5-event-source   |   "created_at": "2026-05-02T22:54:46.558814"
lab5-event-source   | }
```

- docker compose logs event-router

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose logs event-router
lab5-event-router  | Event Router started. Waiting for events...
lab5-event-router  |
lab5-event-router  | Received event: image.uploaded
lab5-event-router  | {
lab5-event-router  |   "event_id": "db615c4a-8559-4b8c-b90a-cc877929e2ed",
lab5-event-router  |   "event_type": "image.uploaded",
lab5-event-router  |   "file_name": "test_image.png",
lab5-event-router  |   "file_path": "/data/input/test_image.png",
lab5-event-router  |   "width": 300,
lab5-event-router  |   "created_at": "2026-05-02T22:54:46.558814"
lab5-event-router  | }
lab5-event-router  | Sent to http://image-resizer:5000/resize
lab5-event-router  | Response code: 200
lab5-event-router  | Duration: 1823.14 ms
lab5-event-router  | Response: {"function":"image-resizer","input_file":"test_image.png","new_size":[300,200],"original_
size":[1200,800],"output_file":"/data/output/resized_test_image.png","processing_ms":1809.32,"status":"success"}
lab5-event-router  |
lab5-event-router  | Sent to http://notifier:5000/notify
lab5-event-router  | Response code: 200
lab5-event-router  | Duration: 9.73 ms
lab5-event-router  | Response: {"event_type":"image.uploaded","file_name":"test_image.png","function":"notifier","status
":"notified","timestamp":"2026-05-02T22:54:48.483055"}
lab5-event-router  |
```

- docker compose logs image-resizer:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose logs image-resizer
lab5-image-resizer  |  * Serving Flask app 'app'
lab5-image-resizer  |  * Debug mode: off
lab5-image-resizer  | WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
lab5-image-resizer  |  * Running on all addresses (0.0.0.0)
lab5-image-resizer  |  * Running on http://127.0.0.1:5000
lab5-image-resizer  |  * Running on http://172.20.0.4:5000
lab5-image-resizer  | Press CTRL+C to quit
lab5-image-resizer  | 172.20.0.6 - - [02/May/2026 22:54:48] "POST /resize HTTP/1.1" 200 -
```

- docker compose logs notifier:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose logs notifier
lab5-notifier  |  * Serving Flask app 'app'
lab5-notifier  |  * Debug mode: off
lab5-notifier  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
lab5-notifier  |  * Running on all addresses (0.0.0.0)
lab5-notifier  |  * Running on http://127.0.0.1:5000
lab5-notifier  |  * Running on http://172.20.0.3:5000
lab5-notifier  | Press CTRL+C to quit
lab5-notifier  | NOTIFICATION: {'status': 'notified', 'function': 'notifier', 'event_type': 'image.uploaded', 'file_name
': 'test_image.png', 'timestamp': '2026-05-02T22:54:48.483055'}
lab5-notifier  | 172.20.0.6 - - [02/May/2026 22:54:48] "POST /notify HTTP/1.1" 200 -
```

14. Step 9 - Test the Function Directly:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> Invoke-RestMethod `
>>   -Method Post `
>>   -Uri http://localhost:5001/resize `
>>   -ContentType "application/json" `
>>   -Body '{"file_path":"/data/input/test_image.png","file_name":"test_image.png","width":200}'


function      : image-resizer
input_file    : test_image.png
new_size      : {200, 133}
original_size : {1200, 800}
output_file   : /data/output/resized_test_image.png
processing_ms : 1850.52
status        : success
```

- The image-resizer function was tested directly using an HTTP request, simulating a Function-as-a-Service (FaaS) invocation.
- A POST request was sent to the /resize endpoint with the image path and target width.
- The function successfully processed the image and returned a JSON response containing metadata such as original size, new size, and processing time.
- The resized image was saved in the output directory, confirming correct function execution.

15. Step 10 - Cold Start Mini-Experiment:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> $start = Get-Date; Invoke-RestMethod -Method Post -Uri http://localho
st:5001/resize -ContentType "application/json" -Body '{"file_path":"/data/input/test_image.png","file_name":"test_image.
png","width":200}'; $end = Get-Date; Write-Host "Total time: $(($end - $start).TotalSeconds)s"


function      : image-resizer
input_file    : test_image.png
new_size      : {200, 133}
original_size : {1200, 800}
output_file   : /data/output/resized_test_image.png
processing_ms : 66.78
status        : success
```

After restarting:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose restart image-resizer
[+] restart 0/1
 - Container lab5-image-resizer Restarting                                                                    2.4s
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> $start = Get-Date; Invoke-RestMethod -Method Post -Uri http://localho
st:5001/resize -ContentType "application/json" -Body '{"file_path":"/data/input/test_image.png","file_name":"test_image.
png","width":200}'; $end = Get-Date; Write-Host "Total time: $(($end - $start).TotalSeconds)s"


function      : image-resizer
input_file    : test_image.png
new_size      : {200, 133}
original_size : {1200, 800}
output_file   : /data/output/resized_test_image.png
processing_ms : 80.32
status        : success

Total time: 0.106965s
```

Measurement	Value
Warm request time	0.089s (88.6ms)
First request after restart	0.107s (106.9ms)
Difference	~0.018s (18ms)
Reason / explanation	The container had already finished restarting before the request was sent, so Flask and Pillow were loaded. The small difference reflects Python re-importing libraries on first use. A larger cold start would be seen if the request was sent *during* restart.

docker compose ps results:

```
PS D:\CMC_labs\lab5\lecture5-local-serverless-lab> docker compose ps
NAME                IMAGE                                          COMMAND                  SERVICE         CREATED
  STATUS         PORTS
lab5-event-router   lecture5-local-serverless-lab-event-router     "python event_router…"   event-router    2 hours ago
  Up 2 hours
lab5-event-source   lecture5-local-serverless-lab-event-source     "python watcher.py"      event-source    2 hours ago
  Up 2 hours
lab5-image-resizer  lecture5-local-serverless-lab-image-resizer    "python app.py"          image-resizer   2 hours ago
  Up 4 minutes   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
lab5-notifier       lecture5-local-serverless-lab-notifier         "python app.py"          notifier        2 hours ago
  Up 2 hours     0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
lab5-redis          redis:7-alpine                                 "docker-entrypoint.s…"   redis           2 hours ago
  Up 2 hours     0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
```

18. Reflection Questions:

1. What is the event source in this lab?

The event source is watcher.py, running inside the event-source container. It watches the /data/input folder and when a new PNG or JPG image is detected, it publishes an image.uploaded event into the Redis Stream.

2. What is the event router?

The event router is event_router.py, running inside the event-router container. It reads events from the Redis Stream and forwards them to the correct function endpoints based on the event type. For image.uploaded events, it routes to both the image-resizer and the notifier.

3. What are the event destinations?

There are two event destinations:

- **Image Resizer** (/resize endpoint on port 5001) — resizes the image and saves it to /data/output
- **Notifier** (/notify endpoint on port 5002) — logs a notification message about the event

4. Why is this pipeline loosely coupled?

Because the event source does not directly call the functions. It only publishes an event to Redis. The router and functions have no knowledge of each other — the router just sends HTTP requests to endpoints. This means any component can be replaced, restarted, or extended without breaking the others.

5. What happened after restarting the image-resizer container?

The container took approximately 2.4 seconds to restart. By the time the first request was sent, Flask had already finished initializing, so the response was only slightly slower than the warm request (0.107s vs 0.089s). The small ~18ms difference reflects the overhead of Python re-loading libraries after restart.

6. How is this similar to serverless cold start?

In real serverless platforms like AWS Lambda, the first request after a function has been idle triggers a cold start — the runtime must be initialized before the request is handled, causing extra latency. In this lab, restarting the container simulates that by forcing the Flask app and Pillow library to reload from scratch before serving the first request.

7. What is missing compared with real cloud serverless platforms? Several things are missing:

- **True ephemerality** — our Flask containers are long-running, not spun up and destroyed per request
- **Auto-scaling** — real platforms scale function instances automatically based on load
- **Sandboxing and isolation** — AWS Lambda uses microVMs (Firecracker) to isolate each function execution
- **Managed infrastructure** — no servers, networking, or container management needed on real platforms
- **Billing per invocation** — real FaaS charges only for actual execution time

8. How could this pipeline be extended to support image moderation, OCR, or ML enrichment?

New function containers could be added and registered as additional destinations in the router's routing table. For example:

- An **image moderation** function could call a content-safety model to flag inappropriate images
- An **OCR function** could use Tesseract to extract text from images and save it alongside the output
- An **ML enrichment function** could run an object detection model and tag the image with detected labels

github link: https://github.com/MennaHesham9/CMC_labs/tree/main/lab5